Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 6:

- Data and Control Flow
- Refute Problems and Debugging



A CSR Initiative by



NPTEL

Example 1: `is_multiple`

- Let us run this code on PythonTutor:

```
1    def is_multiple(n: int, m: int) -> bool:
9        if m == 0:
10           return False
11       return n % m == 0
12
13   a = input('First integer: ')
14   b = input('Second integer: ')
15
16   if is_multiple(a, b):
17       print(f'{a} is a multiple of {b}')
18   else:
19       print(f'{a} is NOT a multiple of {b}')
```

Added by Mypy (**type checker**)

Key observations

- A `def` statement is similar to an assignment
  - The entire body of the function is skipped
  - Body is executed only when the function is called
- Each function parameter is a **local variable**
  - Different to any variable in the **Global frame**, even if the names are identical
  - When the function is called, each parameter is assigned to the corresponding argument
- A local variable **cannot** be accessed outside the function in which it is defined
  - The object it refers to can be `return`-ed

NPTEL

Example 2: `is_multiple` (modified)

- Does the new doctest pass? (OnlineGDB)

```
1    def is_multiple(n: int, m: int) -> bool:
2        """Check if n is an integer multiple
3        of m.
4        >>> is_multiple(10, 5)
5        True
6        >>> is_multiple(10, 0)
7        False
8        >>> is_multiple(0, 0)
9        True
10       """
11       if m == 0:
12           return False
13       return n % m == 0
```

NPTEL

Example 3: `is_multiple` (debugging)

- We now know that the function is **buggy**
  - The input `n=0`, `m=0` is a **counter-example**
- Our friend has attempted to **debug** the function as follows:

```
if m == 0:
    return n == 0

return n % m == 0
```

Nesting

Simplification #1

Instead of:

```
if expression:
    return True
else:
    return False
```

use `return expression`

NPTEL

Refute Problems

- **Goal**: To break the habit of blindly accepting code just because it "looks correct"

  Essential in the Gen AI era!

```
1  # The following function is BUGGY!
2
3  def median(a: int, b: int,
4      """Calculate the median
5      >>> median(1, 2, 3)
6      2
7      >>> median(9, 3, 6)
8      6
9      >>> median(7, 8, 0)
10     7
11     """
12     if a <= b <= c:
13         return b
14     if b <= a <= c:
15         return a
16
17     return c
```

```
19  # Refute the above function by providing the following information:
20
21  # Argument(s):
22  a = ...
23  b = ...
24  c = ...
25
26  # The expected ans                                    on the above arguments)
27  expected_answer =
28
29  # The value actual                                    the above arguments:
30  return_value = ...
31
```

CodeCheck Reset

Simplification #2

After:

```
if expression:
    return value
```

Can replace **elif** with **if**

No need for **else**

Refute Problems: Feedback

- Each time you click CodeCheck, we generate feedback in this order:

```
19  # Refute the above function by providing the following information:
20
21  # Argument(s):
22  a = ...
23  b = ...
24  c = ...
25
26  # The expected answer (the function SHOULD return this on the above arguments)
27  expected_answer = ...
28
29  # The value actually r                                    guments:
30  return_value = ...
31
```

CodeCheck Reset

4. Counter-example?
Examine the input space and the buggy code.

2. Correct expected answer?
Perhaps you misunderstood the task.

3. Correct return value?
Perhaps an error in your **trace**.

NPTEL

Try one!

```
def num_days(month: int, is_leap: bool) -> int:
```

- How many days in a given month?
- Usually 30 or 31
- February: 28 or 29 (leap years)
- Illegal inputs (e.g., `is_leap='Yes'`)

- Link: CodeCheck

NPTEL

Critique Our Friend

```
def num_days(month: int, is_
    if not(1 <= month <= 12)
        return 0
    if month == 2:
        return 29 if is_leap else 28
    if month == 4 or 6 or 9 or 11:
        return 30
    return 31
```

$$\begin{cases} 29, \text{if } \textbf{is_leap} \\ 28, \text{otherwise} \end{cases}$$

PythonTutor

Simplification #3

Replace:

```
if expression:
    var = expr1
else:
    var = expr2
```

with

```
var = expr1 if expression else expr2
```

```
not(1 <= month <= 12)
≡ not(1 <= month and month <= 12)
≡ not(1 <= month) or not(month <= 12)
≡ (1 > month) or (month > 12)
≡ month < 1 or month > 12
```

not(c1 or c2)
= not(c1) and not(c2)



Source: Wikipedia
De Morgan's laws

Short-circuiting (Python docs)

The Boolean operators `and` and `or` are so-called *short-circuit* operators: their arguments are evaluated from left to right, and evaluation stops as soon as the outcome is determined. For example, if `A` and `C` are true but `B` is false, `A and B and C` does not evaluate the expression `C`.

- **Boolean Algebra**: *and*, *or* are **commutative**
  - $(A\ and\ B) \equiv (B\ and\ A)$; $(A\ or\ B) \equiv (B\ or\ A)$
  - $(\text{False}\ and\ X) \equiv \text{False}$; $(\text{True}\ or\ X) \equiv \text{True}$
- Which of these is better?

✓
```
return m > 0 and n % m == 0
```

```
return n % m == 0 and m > 0
```

NPTEL

Summary

- Conditional statements + function calls can lead to complex **control flow** and **data flow**
  - Practice code **tracing**, use PythonTutor to check
- **Refute Problems**: Seemingly correct but **buggy** code
  - **Our goal**: Find a **counter-example** input and show that the **expected answer** ≠ **return value** on this input
  - We can then **debug** the code
- Many equivalent ways of writing code
  - We must be able to **comprehend** common styles

NPTEL